

第15讲：MSE - 为什么要平方？为什么要平均？

Mean Squared Error（均方误差）：从一个 Error 到可优化的回归 Loss

本讲核心问题：为什么要把 Error 平方？为什么再取平均？这两个看似简单的操作，实际上决定了模型如何看待“大错”和“小错”。

学完以后你应该能做到

- 逐项读懂 MSE 公式中的 N、prediction、target、平方和平均。
- 能够对一个小的 Batch 手工算出 MSE，并预测每一步 Shape。
- 理解平方为什么消除正负抵消，并为什么会放大大误差。
- 理解 mean reduction 与 sum reduction 的区别。
- 知道 MSE 的适用场景、局限，以及它和 LLM / Agent Systems 的关系。



1. 从 Error 到 MSE：先回答“为什么需要它”

上一讲我们已经知道，模型先产生 **Prediction**，再与 **Target** 比较。但如果直接使用带正负号的 Error，会发生一个严重问题：正误差和负误差可能互相抵消。

具体数字例子：Target 都是 4，Prediction 是 [2, 4, 6]。那么 Error 是 [-2, 0, +2]，平均 Error 恰好是 0。模型显然并没有全对，但这个指标却给出了“零误差”的假象。

MSE 用两步解决这个问题：

1. **Square：**把每个 Error 平方，使负数和正数都变成非负惩罚。
2. **Mean：**把一批误差压缩成一个可以比较和优化的标量。

$$\text{MSE} = (1 / N) \times \sum (\text{prediction}_i - \text{target}_i)^2$$

通俗解释：每个样本先算“错多少”，把错的程度平方，再把所有样本的平方惩罚取平均。

《深度学习零基础科研基础课》第15讲

2. 公式逐项拆解：每一个符号到底是什么

$$\text{MSE} = (1 / N) \times \sum_{i=1 \dots N} (\hat{y}_i - y_i)^2$$

符号	含义	在代码里常是什么
y_i	第 i 个真实目标 Target	target[i]
$\hat{y}_i$	第 i 个模型预测 Prediction	prediction[i]
$\hat{y}_i - y_i$	带符号 Error	prediction - target
$(...)^2$	Squared Error	(prediction-target)**2
Σ	把所有误差项加起来	np.sum(...)
N	参与平均的误差元素个数	通常是元素数量

常见误区：MSE 中的 N 不是统计学样本方差里的 N-1。这里不是在做“无偏方差估计”，而是在定义一个训练/评估 Loss。

在 PyTorch 的 `nn.MSELoss(reduction="mean")` 中，默认会对输出与 Target 的**所有元素**的平方误差取平均。

3. 手算一个 Batch：不要让公式停留在纸上

设一个回归任务的 Batch 为：

Prediction = [3, 5, 2, 8] Target = [4, 4, 2, 6]

样本	Prediction	Target	Error	Squared Error
1	3	4	-1	1
2	5	4	+1	1
3	2	2	0	0
4	8	6	+2	4

$$MSE = (1 + 1 + 0 + 4) / 4 = 6 / 4 = 1.5$$

Shape: Prediction.shape=(4,)，Target.shape=(4,)，SquaredError.shape=(4,)，最后经过 mean reduction 得到一个标量，shape 可以理解为 ()。

《深度学习零基础科研基础课》第15讲

4. 为什么要“平方”：它一次解决两个问题

问题 A：消除正负抵消

Error=-2 与 Error=+2 平方以后都变成 4，因此不会互相抵消。

问题 B：让大误差受到更重惩罚

Error 从 1 增大到 2，绝对误差只从 1 变 2，但平方误差从 1 变 4；Error=10 时平方误差达到 100。



科研直觉：选择平方并不是“数学规定”。它意味着你的 Objective 在表达一种偏好：大误差应该被超线性地惩罚。如果任务不希望单个异常值支配训练，MSE 可能并不是最佳选择。

《深度学习零基础科研基础课》第15讲

5. 为什么要“平均”：让 Loss 更容易跨 Batch 比较

如果只用 Sum of Squared Error，那么完全相同的数据复制一遍，Loss 会直接翻倍。

	Batch Size	Sum	Mean
原 Batch	4	6	1.5
完全复制一次	8	12	1.5

因此 mean reduction 让 Loss 在“只是 Batch Size 改变”时更具有可比性。

但要注意：这不意味着改 Batch Size 后训练动力学完全不变。Batch Size 会改变梯度噪声、更新次数、统计性质等；这里只是在解释 Loss 数值的缩放。

mean = sum / number_of_error_elements

《深度学习零基础科研基础课》第15讲

6. 多输出与 Shape：到底是在对谁求平均？

如果每个样本不是一个标量输出，而是一个长度为 3 的向量：

```
Prediction.shape = (2, 3)    Target.shape = (2, 3)
```

逐元素平方以后仍然是：

```
SquaredError.shape = (2, 3)
```

此时至少存在两种合理的 reduction：

- **每个样本自己的 MSE**：沿最后一维 mean，得到 shape=(2,)。
- **整个 Batch 的全局 MSE**：对全部 6 个元素 mean，得到一个标量。

本讲实验得到：两个样本的 per-sample MSE 都是 1.6667，因此 global MSE 也为 1.6667。

读论文时要问：作者说“使用 MSE”仍然不够。必须看它是在哪些维度上 reduction，是否对不同任务、时间步、特征维使用了不同权重。

《深度学习零基础科研基础课》第15讲

7. MSE 为什么容易被异常大误差支配

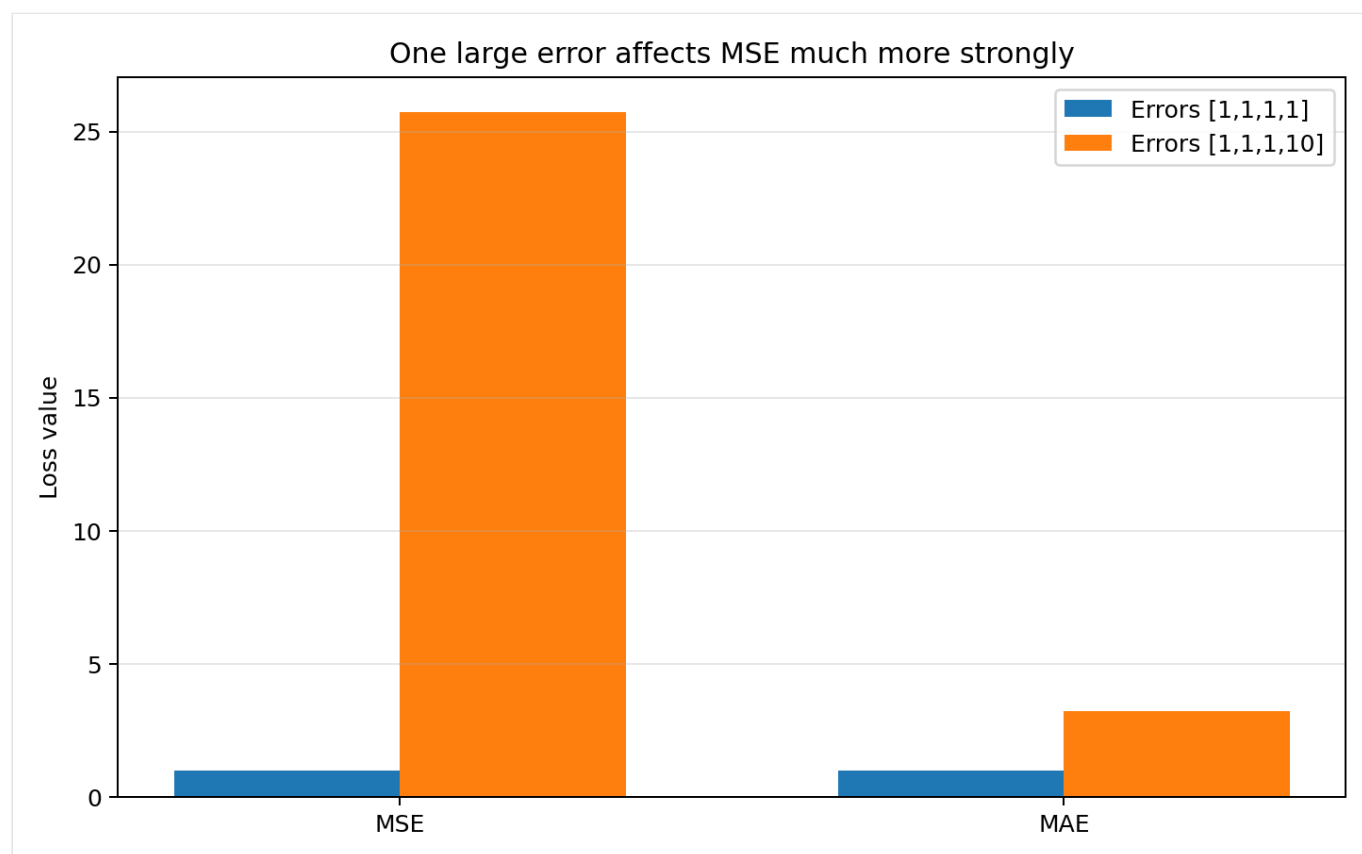
对 $\text{Error}=[1,1,1,1]$:

$$\text{MSE} = 1.0$$

只把最后一个误差改成 10:

$$\text{Error}=[1,1,1,10] \rightarrow \text{MSE}=25.75$$

作为对照, MAE 只从 1.0 上升到 3.25。



底层原因: 10 的平方是 100, 一个大误差项可以压过许多小误差项。

不要机械认为“对异常值敏感 = 坏”。如果业务确实认为一次巨大错误代价极高, 这种敏感性恰好可能是你需要的。

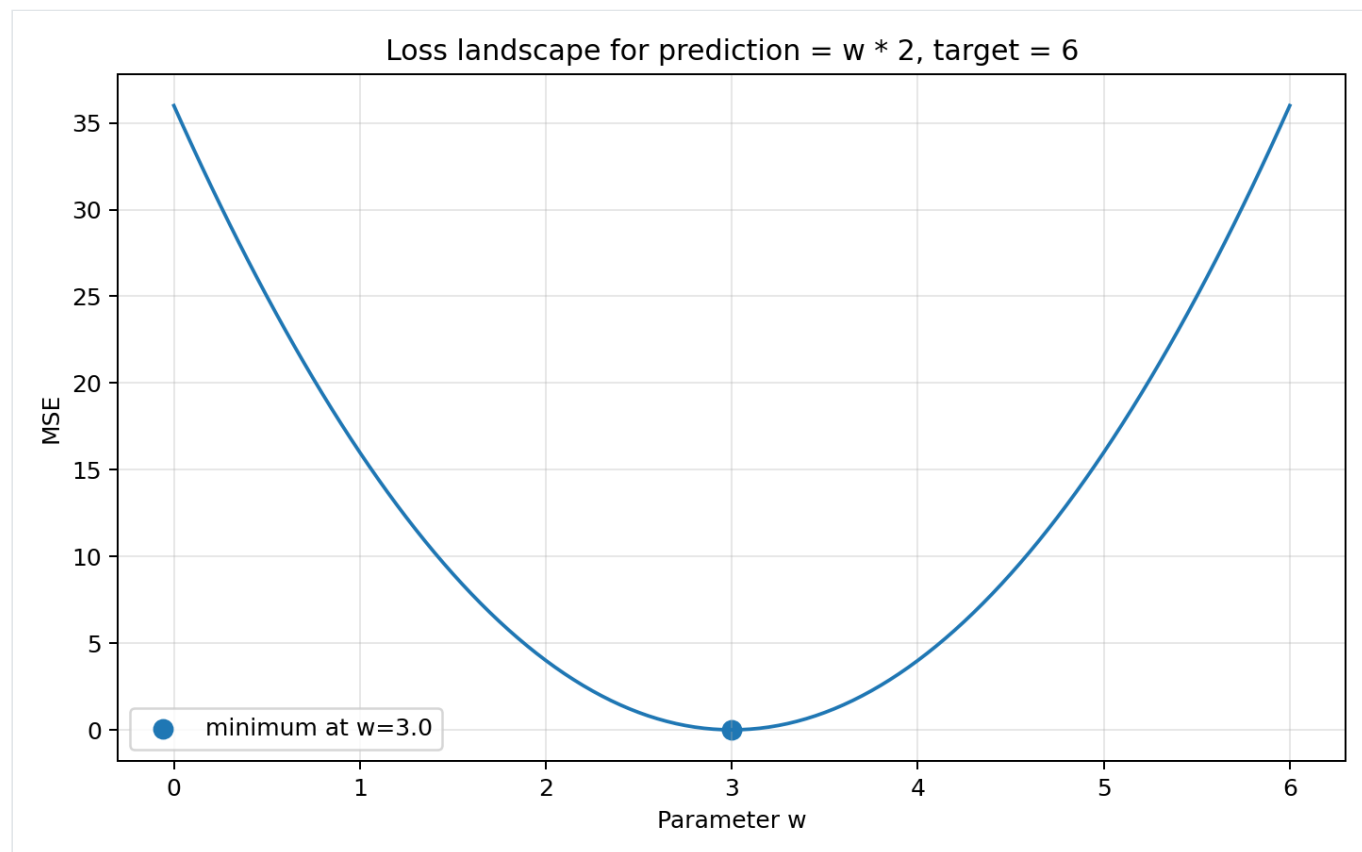
8. MSE 与 Loss Landscape: 参数为什么会有“下坡方向”

设只有一个参数 w :

$$\text{prediction} = w \times 2 \quad \text{target} = 6$$

于是:

$$\text{MSE}(w) = (2w - 6)^2$$



曲线最低点在 $w=3$, 因为此时 $\text{prediction}=6$, 与 target 完全一致, $\text{MSE}=0$ 。

当 $w=2$ 时, $\text{MSE}=4$ 。本讲数值导数实验得到 $dL/dw=-8$ 。负号表示: 如果 w 稍微增大, Loss 会下降。

这里已经把第13讲的“导数是局部敏感度”和本讲的“Loss 是优化目标”连接起来了。后面 Gradient Descent 就是在系统地利用这种方向信息。

《深度学习零基础科研基础课》第15讲

9. NumPy 与 PyTorch：框架帮你做的其实很少

手写 NumPy：

```
error = prediction - target
squared_error = error ** 2
loss = np.mean(squared_error)
```

PyTorch：

```
criterion = nn.MSELoss(reduction="mean")
loss = criterion(prediction, target)
```

底层概念没有改变：**相减** → **平方** → **reduction**。

本讲实测：NumPy mean MSE=1.5000，PyTorch mean=1.5000；NumPy sum=6.0000，PyTorch sum=6.0000。

后面进入 Autograd 后，PyTorch 的真正价值不是“会算 MSE”，而是它能够记录计算图并自动计算 Loss 对大量参数的梯度。

《深度学习零基础科研基础课》第15讲

10. MSE 与 LLM / Transformer / Agent Systems 的关系

你以前在 LLM 课程中见过 Loss，但 **Next-token Prediction** 的主 Loss 通常不是 MSE，而是 **Cross Entropy**。原因是下一个 Token 是一个离散类别选择问题，而不是普通连续数值回归。

那么为什么我们仍然必须认真学 MSE？因为它是理解 Loss、Gradient 和优化的最干净入口，而且在很多 AI 系统子任务中仍然非常常见：

- 连续数值回归，例如评分、价值、时间、距离或资源预测。
- 某些表示对齐、蒸馏或辅助 Objective 中，对连续向量差异做平方惩罚。
- Agent Systems 中，如果某个模块预测连续的 cost / reward / latency / confidence surrogate，也可能采用回归损失。

科研判断：不要因为一个 Loss “常见” 就默认它合理。Loss 实际上定义了模型训练时被奖励和惩罚的方式。论文改变 Loss，本质上可能是在改变研究目标。

《深度学习零基础科研基础课》第15讲

11. 常见误区与科研读论文检查表

误区	正确理解
MSE 越小，模型一定越好	只说明指定数据与指定 Objective 下平方误差较小；还要看泛化、分布、指标与实际目标。
MSE=0 就代表模型学会规律	可能只是训练样本拟合为 0；不能推出测试集也为 0。
MSE 对异常值敏感，所以一定不好	是否合理取决于任务是否希望重罚大误差。
mean 和 sum 只是写法不同	会改变 Loss 和梯度的数值尺度，进而影响优化超参数。
论文说用了 MSE 就足够复现	还要确认 reduction 维度、mask、sample weighting、normalization 等。
MSE 的单位与原目标相同	MSE 的单位被平方；若需要原单位直觉，可看 RMSE。

最低闭环：看到 MSE 时，先问 5 件事：谁和谁相减？Shape 是什么？在哪些维度平方？在哪些维度求平均？大误差被怎样加权？

《深度学习零基础科研基础课》第15讲

12. 本讲总结与下一讲连接

$$\text{MSE} = \text{mean}((\text{prediction} - \text{target})^2)$$

- **Error**: 保留方向，但正负可能抵消。
- **Square**: 消除抵消，并让大误差受到更强惩罚。
- **Mean**: 把多个误差项压缩成一个标量，并减少 Batch Size 对 Loss 绝对尺度的直接影响。
- **Shape**: 平方通常不改变 Shape，reduction 才会减少维度。
- **优化**: 把 MSE 看成参数的函数，就形成了 Loss Landscape。

下一讲: 为了理解分类 Loss 和 LLM 的 Next-token Prediction，我们需要补最小概率数学：概率、指数 e^x 和对数 $\log$ 。然后才能真正理解 Softmax 和 Cross Entropy。

《深度学习零基础科研基础课》第15讲